

Nombre del equipo

Modcast Team

Autores

- David Javier Montes Fernández - david.j.montes - deivisi
- Manuel Santamarina Ruiz de León - manuel.santamarina - manuelsrleon
- Roi Millán Míguez - roi.millan.miguez - roimm1
- Pablo Masián Carro - pablo.masian.carro - pablomasian
- Antonio Pernas González - antonio.pernasg - antonioPernas

Descripción de la aplicación

Modcast es un sistema P2P descentralizado de sincronización automática de mods (modificaciones) para videojuegos multijugador, desarrollado en Elixir. El sistema permite que los jugadores compartan contenido personalizado (modelos 3D, texturas, skins) sin necesidad de servidores centrales, facilitando a los desarrolladores la integración de contenido generado por usuarios en sus juegos.

Requisitos Funcionales

1. Gestión de Sesiones P2P

- Iniciar sesión como host
- Unirse a sesión existente
- Desconectarse de sesión
- Mantener lista de jugadores conectados

Formato

- Se utiliza `mix format` con la configuración estándar de Elixir.
- Indentación: 2 espacios.
- Longitud máxima de línea: 98 caracteres.

Documentación

- Todos los módulos públicos deben incluir `@moduledoc`.
- Todas las funciones públicas deben incluir `@doc`.
- Se utiliza ExDoc para generar documentación HTML.
- Formato: Markdown para documentación.

Guía de estilo

Se sigue [The Elixir Style Guide](#):

- Nombres de módulos: PascalCase (`Modcast.GameStateComponent`)
- Nombres de funciones: snake_case (`put_entity/2`)
- Nombres de variables: snake_case (`player_id`)
- Constantes: módulos de atributo (`@game_port 5050`)
- Pattern matching preferido sobre condicionales
- Uso de pipe operator `|>` para cadenas de transformación
- Guards para validación de tipos cuando sea apropiadoe Entidades (`Sync Status Ledger`)**

- Crear entidades vinculadas a mods
- Transferir propiedad de entidades entre jugadores
- Mantener registro sincronizado entre todos los peers
- Consultar entidades por jugador/mod/asset

4. Interfaz con Motor de Juego (MSAPI)

- Comunicación TCP/JSON
- Comandos: start_session, join_session, select_mods, register_entity, start_game
- Eventos asíncronos: mod_ready, game_started, entity_created

Requisitos No Funcionales

1. Disponibilidad

- Arquitectura P2P sin punto único de fallo
- Resistencia a desconexiones de peers
- Reconexión automática

2. Rendimiento

- Límite de tamaño de archivo: 10 MB por mod
- Transferencias simultáneas entre múltiples peers
- Detección de mods duplicados por hash

3. Consistencia

- Resolución de conflictos mediante Last Write Wins (LWW)
- Versionado del Sync Status Ledger
- Sincronización periódica automática

4. Interoperabilidad

- Interfaz agnóstica al motor de juego
- Protocolo JSON sobre TCP
- Formato de mods: archivos ZIP estándar

5. Seguridad

- Validación de integridad mediante MD5
- Validación de sesión en transferencias
- Verificación de formato de archivos

Información a incluir en la documentación del proyecto

Normas para el código

- Documentar el código con *ExDoc*.
- Documentar las normas para el formato del código. Recomendación: usar `mix format` y documentar la configuración del mismo.
- Crear, y cumplir, una guía de estilo para el código. Recomendación: https://github.com/christopheradams/elixir_style_guide.

Normas para el proyecto y el control de versiones

Estructura del proyecto

Basada en la estructura estándar de Mix:

```
modcast/
├── lib/
│   └── modcast/
│       ├── msapi.ex                # Interfaz con motor de juego
│       ├── game_state/
│       │   ├── game_state_component.ex # Orquestador P2P
│       │   └── entity.ex              # Entidades del juego
│       ├── file_transfer_component.ex # Transferencia de archivos
│       ├── sync_status_ledger.ex      # Registro distribuido
│       ├── persistence.ex             # Capa de persistencia
│       └── utils.ex                   # Utilidades
├── test/
│   ├── sync_status_ledger_test.exs
│   └── test_helper.exs
├── demo/
│   └── car-sumo/                  # Juego demo en Godot
├── tests/                         # Tests de integración Python
├── mix.exs                         # Configuración del proyecto
├── README.md
└── documentation.md               # Documentación técnica
completa
```

Mensajes de commit

Formato: Para los commits decidimos hacerlos todos en *ingles* y utilizar los verbos en el infinitivo (en vez de Added → Add)

Estrategia de ramas

Feature Branch Workflow:

- **main**: Rama principal estable (releases)
- **develop**: Rama de integración
- **F##-feature-name**: Ramas de features individuales

Proceso:

1. Crear rama desde **develop**: `git checkout -b F04-SSL-implementation`
2. Desarrollar y hacer commits
3. Merge a **develop** cuando esté completo
4. **develop** se mergea a **main** para releases

Ramas del proyecto:

- F01: Godot project initialization
- F02: GSC implementation
- F03: MSAPI Initialization
- F04: SSL implementation
- F08: MDF player car definition
- FD-Memoria

Documentación de la aplicación

Aplicación

Casos de Uso Principales

UC1: Compartir mods en sesión multijugador

- Actor: Jugadores (2-N)
- Flujo:
 1. Player1 inicia sesión con mod personalizado
 2. Player2 se une a la sesión
 3. Sistema detecta mod faltante en Player2
 4. Transferencia automática P2P del mod
 5. Verificación de integridad (MD5)
 6. Ambos jugadores pueden ver el contenido personalizado

UC2: Registrar entidad con mod en el juego

- Actor: Motor de juego
- Flujo:
 1. Juego crea objeto (ej: coche del jugador)
 2. Vincula objeto a mod mediante `register_entity`
 3. SSL registra la relación entity-player-mod
 4. Broadcast a todos los peers
 5. Peers cargan el mod correcto para ese objeto

UC3: Transferir propiedad de entidad

- Actor: Motor de juego
- Flujo:
 1. Evento del juego (ej: jugador abandona vehículo)
 2. Llamada a `transfer_entity` con nuevo propietario
 3. SSL actualiza registro con timestamp
 4. Sincronización con todos los peers
 5. Juego actualiza lógica de propiedad

Diseño

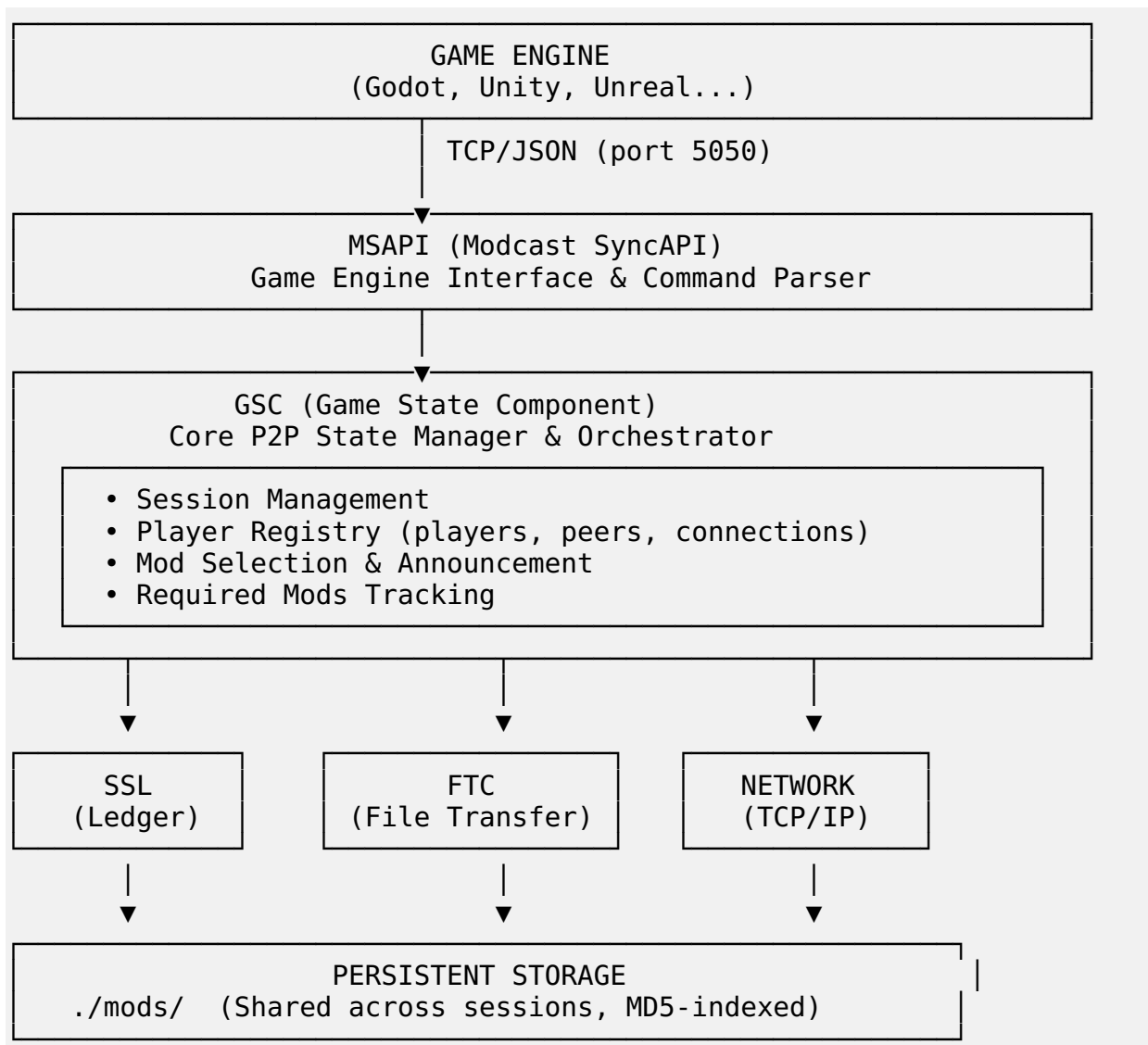
Arquitectura Principal

Estilo arquitectónico: P2P (Peer-to-Peer) con componentes de Event-Driven Architecture

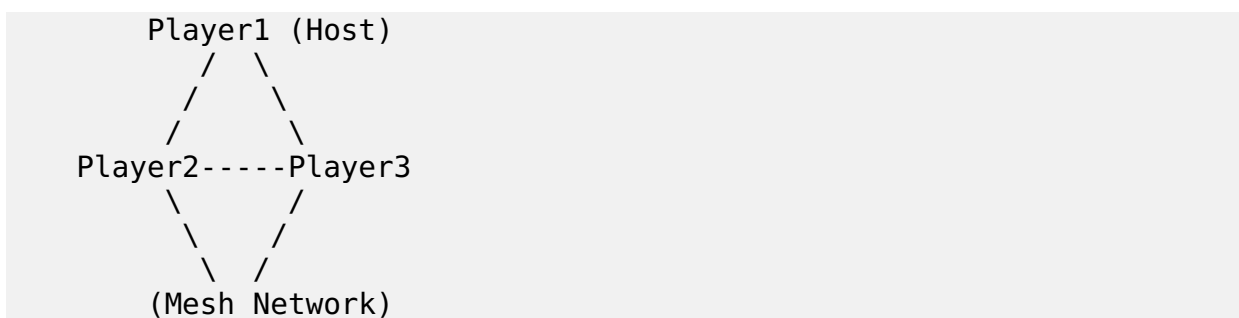
Componentes principales:

1. **MSAPI** - Interfaz JSON/TCP con motor de juego (puerto 5050)
2. **GSC (Game State Component)** - Orquestador P2P (GenServer)
3. **SSL (Sync Status Ledger)** - Registro distribuido CRDT-like
4. **FTC (File Transfer Component)** - Transferencia de archivos
5. **Persistence** - Almacenamiento local DETS (no en uso actualmente)
6. **MMF (Modcast Mod File)** - Formato de mods (.zip)
7. **Storage** - Sistema de archivos persistente (./mods, ./data)

Diagrama de Arquitectura:



Topología de Red:



Cada jugador mantiene conexiones directas con todos los demás

Detalles de Componentes

1. MSAPI (Modcast SyncAPI)

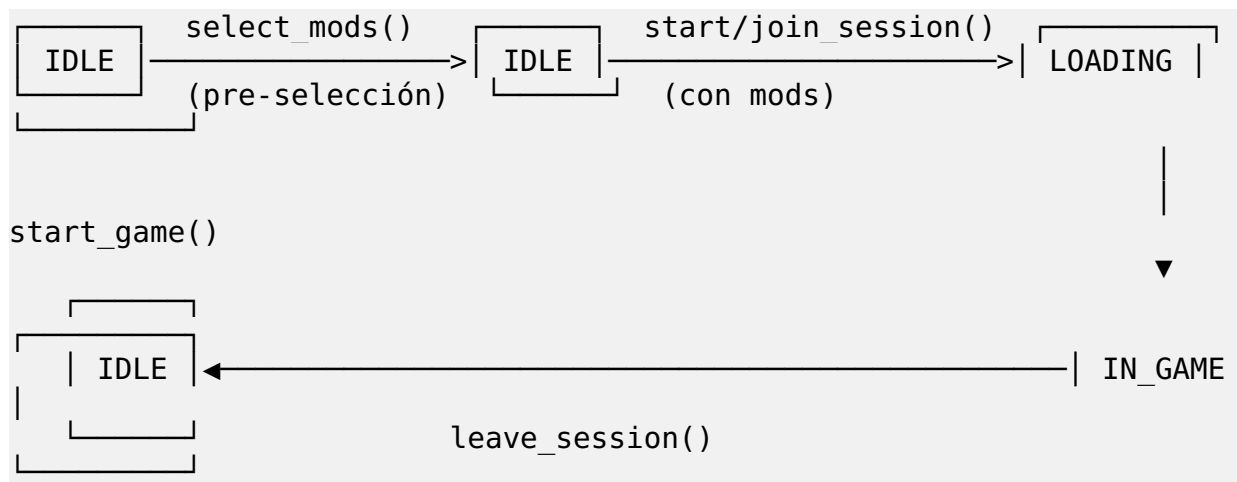
- Ubicación: `lib/modcast/msapi.exe`
- Puerto: 5050 (TCP)

- Protocolo: JSON sobre TCP
- Comandos disponibles:
 - `start_session`: Crear sesión P2P (host)
 - `join_session`: Unirse a sesión existente
 - `select_mods`: Seleccionar mods para compartir
 - `start_game`: Iniciar juego
 - `register_entity`: Crear entidad con mod
 - `transfer_entity`: Transferir propiedad
 - `list_available_mods`: Listar mods locales

2. GSC (Game State Component)

- Ubicación: `lib/modcast/game_state/game_state_component.ex`
- Tipo: `GenServer`
- Responsabilidades:
 - Gestión de sesiones P2P
 - Registro de jugadores
 - Coordinación de transferencias
 - Sincronización de estado

Ciclo de Vida de Sesión:



Flujo de Selección de Mods:

1. Escanear carpeta local → `available_mods`
2. PRE-SELECCIONAR mods en `:idle` (antes de sesión)
3. Unirse/iniciar sesión (válida selección)
4. Anunciar SOLO `selected_mods` a peers
5. Calcular `required_mods` (unión de selecciones)
6. Descargar mods faltantes automáticamente
7. Iniciar juego cuando todos estén listos

3. SSL (Sync Status Ledger)

- Ubicación: `lib/modcast/sync_status_ledger.ex`
- Propósito: Registro distribuido de entidades
- Estructura de entidad:

```
%Entity{
  entity_id: "sword_001",
```

```

asset_id: "weapon_sword",
hash: "abc123...",
player_id: "Player1",
created_at: ~U[2025-01-15 10:00:00Z],
last_transferred: ~U[2025-01-15 10:05:00Z]
}

```

4. FTC (File Transfer Component)

- Ubicación: `lib/modcast/file_transfer_component.ex`
- Protocolo:
 - `{:request_mod, hash, filename, requestor_id}`
 - `{:mod_file, session_id, hash, filename, binary_data}`
- Límite: 10 MB por archivo
- Verificación: MD5 hash antes y después

5. MMF (Modcast Mod File)

- Formato: Archivos ZIP estándar
- Identificación: Hash MD5 del archivo completo
- Contenido: Assets del juego (texturas, modelos, etc.)
- Ejemplo:

```

fire_sword.zip (3.2 MB)
├── textures/
│   ├── blade_fire.png
│   └── handle_wood.png
├── models/
│   └── sword.obj
└── metadata.json

```

6. Almacenamiento Persistente

- Ubicación: `./mods/` (mods descargados)
- Persistencia: Entre sesiones
- Beneficios:
 - Sin descargas redundantes
 - Biblioteca de mods crece con el tiempo
 - Inicio de sesión más rápido

Decisiones de Diseño (ADR)

ADR-001: Arquitectura P2P sin servidor central

- Contexto: Evitar costos de servidor y punto único de fallo
- Decisión: Arquitectura P2P pura con mesh network
- Consecuencias: Mayor complejidad en sincronización, pero mayor disponibilidad

ADR-002: Protocolo JSON sobre TCP

- Contexto: Necesidad de interoperar con múltiples motores de juego
- Decisión: API basada en JSON sobre TCP (puerto 5050)
- Consecuencias: Facilita integración pero sacrifica algo de rendimiento

ADR-003: Hash MD5 para identificación de mods

- Contexto: Necesidad de identificar archivos únicamente

- Decisión: Usar MD5 hash del archivo completo
- Consecuencias: Rápido y suficiente para el scope del proyecto

ADR-004: Resolución de conflictos LWW (Last Write Wins)

- Contexto: Conflictos en SSL cuando peers editan simultáneamente
- Decisión: El cambio más reciente (por timestamp) prevalece
- Consecuencias: Simple y determinista, pero puede perder datos

ADR-005: GenServer para gestión de estado P2P

- Contexto: Necesidad de manejar estado compartido y concurrencia
- Decisión: Usar GenServer para GSC y MSAPI
- Consecuencias: Modelo de actores facilita manejo de mensajes P2P

Tácticas Aplicadas

Disponibilidad:

- Redundancia: Mesh P2P (sin SPOF)
- Heartbeat: Detección de peers desconectados
- Exception handling: Supervisión con restart strategies

Rendimiento:

- Caching: Almacenamiento persistente de mods en `./mods`
- Deduplicación: Hash-based detection de archivos duplicados
- Concurrencia: Task.Supervisor para conexiones simultáneas

Consistencia:

- Versionado: Contador incremental en SSL
- Conflict resolution: LWW basado en timestamps
- Sincronización periódica: Full sync broadcasts

Seguridad:

- Validación de integridad: MD5 hash checking
- Session validation: Verificar session_id en transfers
- Input validation: Pattern matching y guards

Instrucciones

Requisitos previos

- Elixir 1.12 o superior
- Erlang/OTP 24 o superior
- Python 3.8+ (para tests de integración)
- Godot 4.x (opcional, para demo)

Compilación

```
# Clonar repositorio
git clone https://github.com/[team]/modcast.git
cd modcast

# Instalar dependencias
mix deps.get
```



```
# Compilar
mix compile
```

Ejecución

Iniciar servidor Modcast:

```
# Terminal 1 - Player 1
iex -S mix

# En la consola IEx
iex> Modcast.MSAPI.start_link()
```

Integración con juego:

```
import socket
import json

# Conectar al puerto MSAPI
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect(("127.0.0.1", 5050))

# Iniciar sesión
cmd = {"action": "start_session", "session": "game_123", "player":
"Player1"}
sock.send((json.dumps(cmd) + "\n").encode())

# Seleccionar mods
cmd = {"action": "select_mods", "player": "Player1", "hashes":
["abc123"]}
sock.send((json.dumps(cmd) + "\n").encode())
```

Tests

Tests unitarios (Elixir):

```
# Todos los tests
mix test

# Tests específicos
mix test test/sync_status_ledger_test.exs

# Con coverage
mix test --cover
```

Tests de integración (Python):

```
cd tests
python multiple_instance_2.py
```

Tests implementados:

1. **Test 1:** Basic Mod Sharing - 2 jugadores, 1 mod cada uno
2. **Test 2:** Multiple Mods - Múltiples mods por jugador
3. **Test 3:** Three Players - Red mesh de 3 jugadores

Cobertura:

- Sync Status Ledger: ~95% (21/21 tests passing)
- File Transfer: ~70% (tests básicos)
- MSAPI: ~60% (comandos principales)
- GSC: ~50% (lógica P2P básica)

Escenarios cubiertos: Creación y gestión de sesiones P2P Selección y anuncio de mods Transferencia de archivos entre peers Verificación de integridad (MD5) Resolución de conflictos en SSL Persistencia de mods descargados Registro y transferencia de entidades Serialización/deserialización JSON Detección de mods duplicados

Escenarios NO cubiertos: Tests de red real con múltiples máquinas Tests de rendimiento bajo carga NAT traversal (STUN/TURN) Archivos > 10MB (chunked transfer) Reconexión automática tras desconexión

Demo con Godot

```
# 1. Abrir Godot
godot demo/car-sumo/project.godot

# 2. En otra terminal, iniciar Modcast
iex -S mix

# 3. Ejecutar juego desde Godot
# El juego se conectará automáticamente al puerto 5050
```

Protocolo de Red

Mensajes del Sistema

Gestión de Sesiones:

```
{:handshake, session_id, player_id}
{:handshake_response, player_id}
{:handshake_response_with_peers, player_id, peer_list}
{:new_peer_joined, player_id, port}
{:player_left, player_id}
```

Sincronización de Mods:

```
{:selected_mods, player_id, [hash1, hash2, ...]}
{:request_mod, hash, filename, requestor_id}
{:mod_file, session_id, hash, filename, binary_data}
```

Gestión de Entidades:

```
{:entity_created, %Entity{}}
{:entity_transferred, entity_id, new_player_id}
{:full_sync, %SyncStatusLedger{}}
```

Control de Juego:

```
{:game_started}
```

Configuración

Puertos

- **MSAPI:** 5050 (Game Engine ↔ Modcast)
- **P2P Network:** 4040 (Player ↔ Player)

Límites de Archivos

- **Tamaño máximo de mod:** 10 MB (configurable)
- **Formatos soportados:** .zip

Directorios

- **Mods:** ./mods/ (almacenamiento persistente)
- **Data:** ./data/ (estado DETS - no en uso)

Problemas Conocidos y Soluciones

SOLUCIONADO: Hash Mismatch entre Python y Elixir

Problema: Python calculaba hashes MD5 diferentes a Elixir. **Solución:** Los tests ahora consultan a Elixir por los hashes reales.

SOLUCIONADO: Flujo Incorrecto de Selección de Mods

Problema: Mods seleccionados después de unirse, carpeta completa expuesta. **Solución:** Pre-selección implementada en fase :idle.

SOLUCIONADO: Errores de File Handle en Tests

Problema: I/O operation on closed file al reutilizar instancias. **Solución:** Llamar `setup()` al inicio de cada test.

Troubleshooting

Error: "Cannot select mods we don't have"

Causa: Hash mismatch o mods no escaneados **Solución:**

1. Crear mods ANTES de iniciar Elixir
2. Consultar: `list_available_mods`
3. Usar hashes devueltos para `select_mods`

Error: "No mods selected"

Causa: Intentar start/join sin pre-selección **Solución:**

```
# ORDEN CORRECTO:  
select_mods([hash1, hash2]) # Primero
```

```
start_session() # Segundo
```

Error: "Player ID mismatch"

Causa: Diferente player_id en select_mods vs start/join **Solución:** Usar mismo player_id para ambas operaciones

Mejoras Futuras

- ☐ Uso de persistence.ex para recuperación de sesiones
 - ☐ Chunked file transfer para mods >10MB
 - ☐ Compresión (gzip) para transferencias
 - ☐ NAT traversal (STUN/TURN)
 - ☐ Web UI para gestión de mods
 - ☐ Bandwidth throttling
 - ☐ Colas de prioridad para descargas
 - ☐ Versionado de mods
 - ☐ Checksums alternativos (SHA-256)
 - ☐ Actualizaciones automáticas de mods
 - ☐ Gestión de dependencias entre mods
-

Recursos Adicionales

Documentación completa: Ver `documentation.md` para detalles técnicos exhaustivos

Supervisión: David Cabrero Souto

Licencia: GNU General Public License v3.0